Unidad 5
Patrones de comportamiento: cómo se reparten el trabajo los objetos

Prof: Lic. Rosales Pablo (prosales@unpata.edu.ar)
Materia: Programación Orientada a Objetos
Año: 2026

Sacar la regla o la acción a un objeto propio, y que el contexto le delegue.

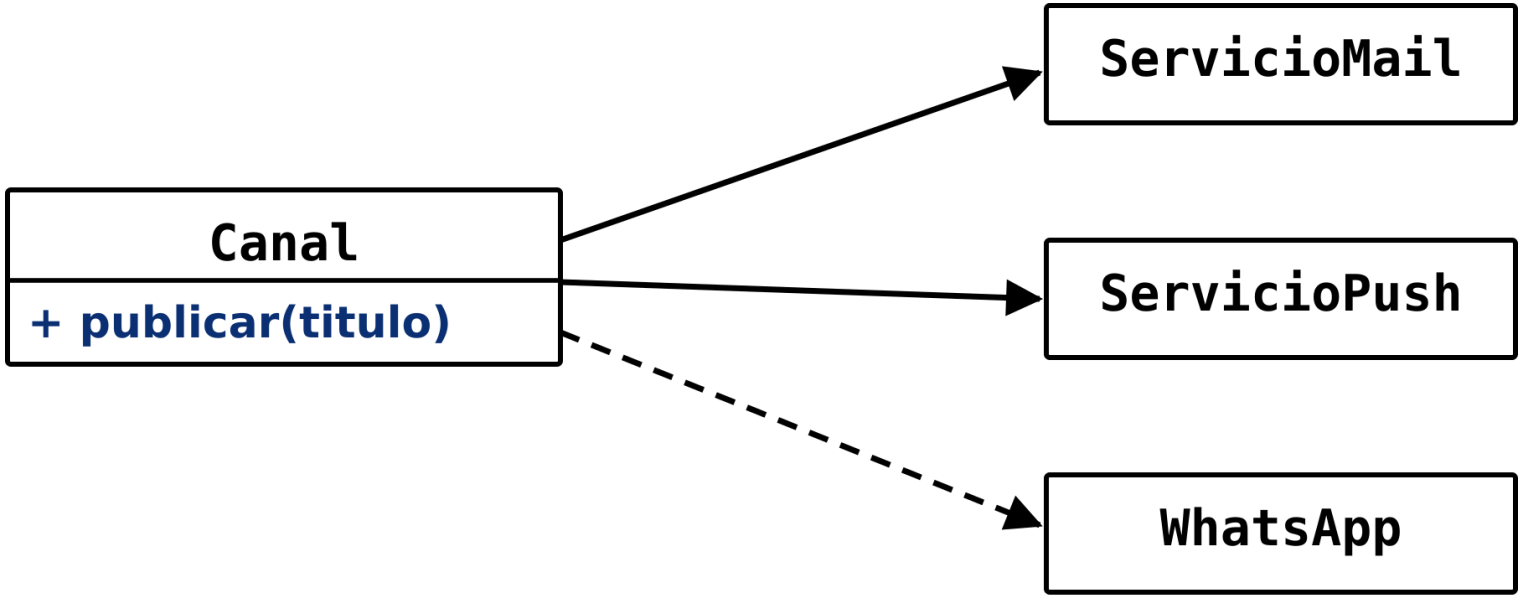
Lo que distingue a cada patrón es qué se convierte en objeto y quién lo maneja.

Lo que más cambia son las reglas

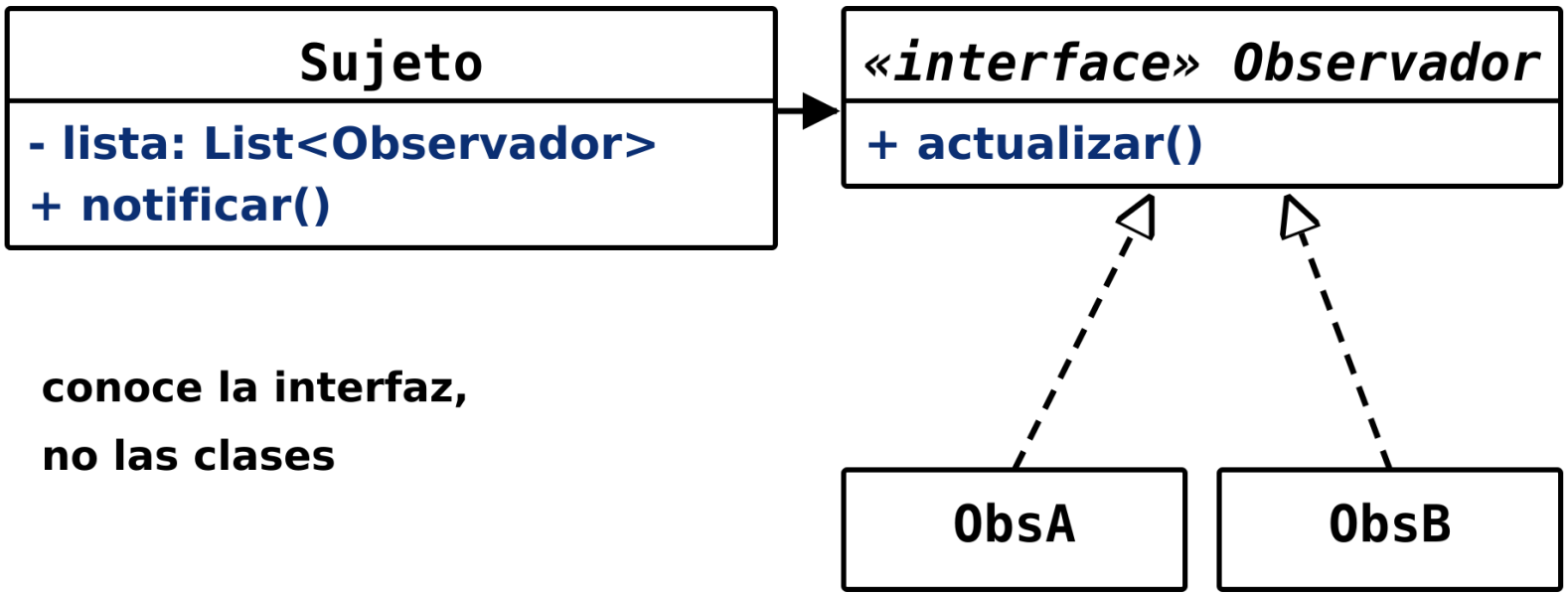
- **Observer**: el que cambia avisa a una lista de interesados.
- **Command**: el pedido es un objeto, se guarda y se deshace.
- **State**: el objeto cambia de comportamiento solo, según su etapa.
- **Strategy**: el comportamiento lo elige quien usa al objeto.

Observer ya lo usaron con la Caja. Command, State y Strategy son nuevos.

Observer: el problema



Observer: el sujeto avisa a su lista



conoce la interfaz,

no las clases

Observer: el canal

```
public class Canal {
  private List<Suscriptor> suscriptores;

  public void publicar(String titulo) {
    for (Suscriptor s : suscriptores)
      s.nuevoVideo(titulo);
  }
}
```

`Suscriptor` es una interfaz con un solo método: `nuevoVideo(titulo)`.

Observer: quién se suscribe

```
Canal canal = new Canal();
canal.suscribir(new AvisoPorMail("ana"));
canal.suscribir(new AvisoPush());

canal.publicar("Patrones");
// llegan el mail y la notificacion
```

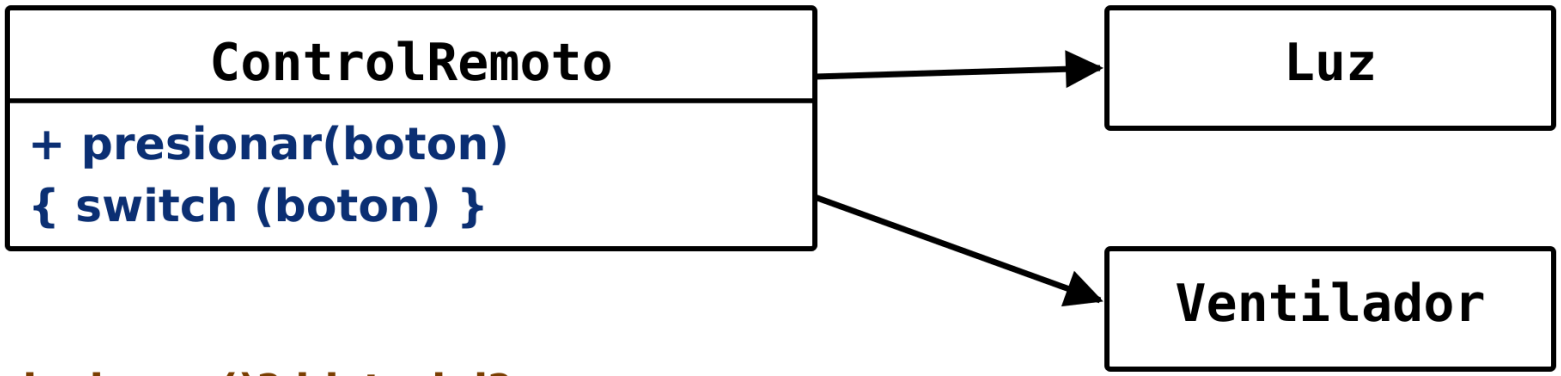
El canal no conoce el mail ni el push. WhatsApp es una clase nueva y una línea.

Observer en el proyecto

- ¿Qué ventanas o paneles tienen que reflejar la partida cuando cambia?
- El tablero, el marcador y las barras de vida observan a la partida o a las unidades.

El modelo avisa; la vista se registra y lee. El proyecto ya lo pide.

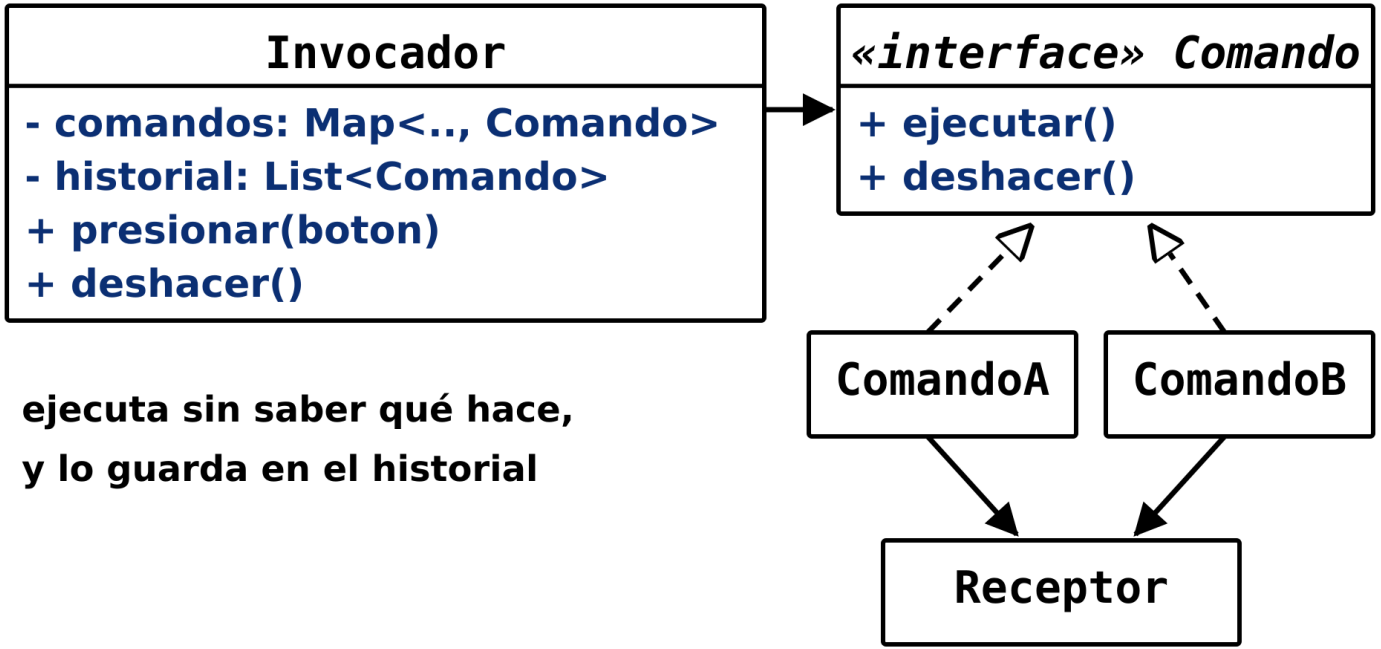
Command: el problema



deshacer()? historial?

la acción no quedó guardada en ningún lado

Command: el pedido es un objeto



ejecuta sin saber qué hace,

y lo guarda en el historial

Command: el comando

```
public interface Comando {
  void ejecutar();
  void deshacer();
}

public class PrenderLuz implements Comando {
  private Luz luz;
  public void ejecutar() { luz.prender(); }
  public void deshacer() { luz.apagar(); }
}
```

Command: el control remoto

```
public class ControlRemoto {
  private Map<Integer, Comando> botones;
  private List<Comando> historial;

  public void presionar(int boton) {
    Comando c = botones.get(boton);
    c.ejecutar();      // no sabe que hace
    historial.add(c);
  }
}
```

Command: deshacer

```
public void deshacer() {
  int ultimo = historial.size() - 1;
  historial.remove(ultimo).deshacer();
}

control.asignar(1, new PrenderLuz(living));
control.presionar(1);    // se prende
control.deshacer();      // se apaga
```

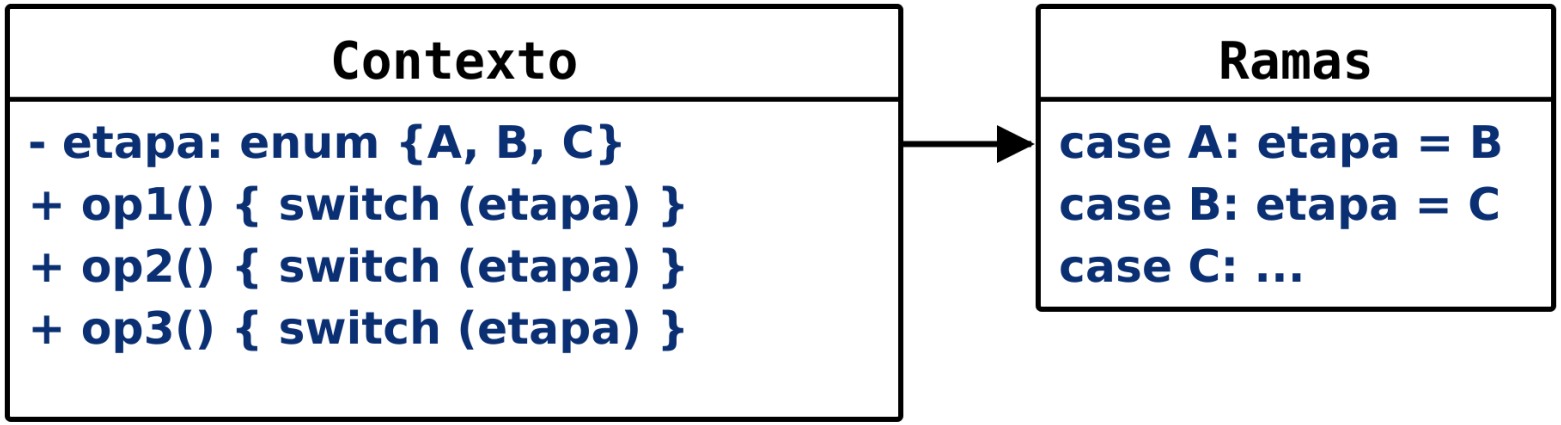
Como el pedido es un objeto, se guarda. Como la lambda de `invokeLater()`.

Command en el proyecto

- ¿Qué acciones dispara el jugador, y qué hay que recordar de ellas?
- Mover, atacar, pasar turno: cada botón tiene su comando.
- El registro de combate es el historial de comandos.

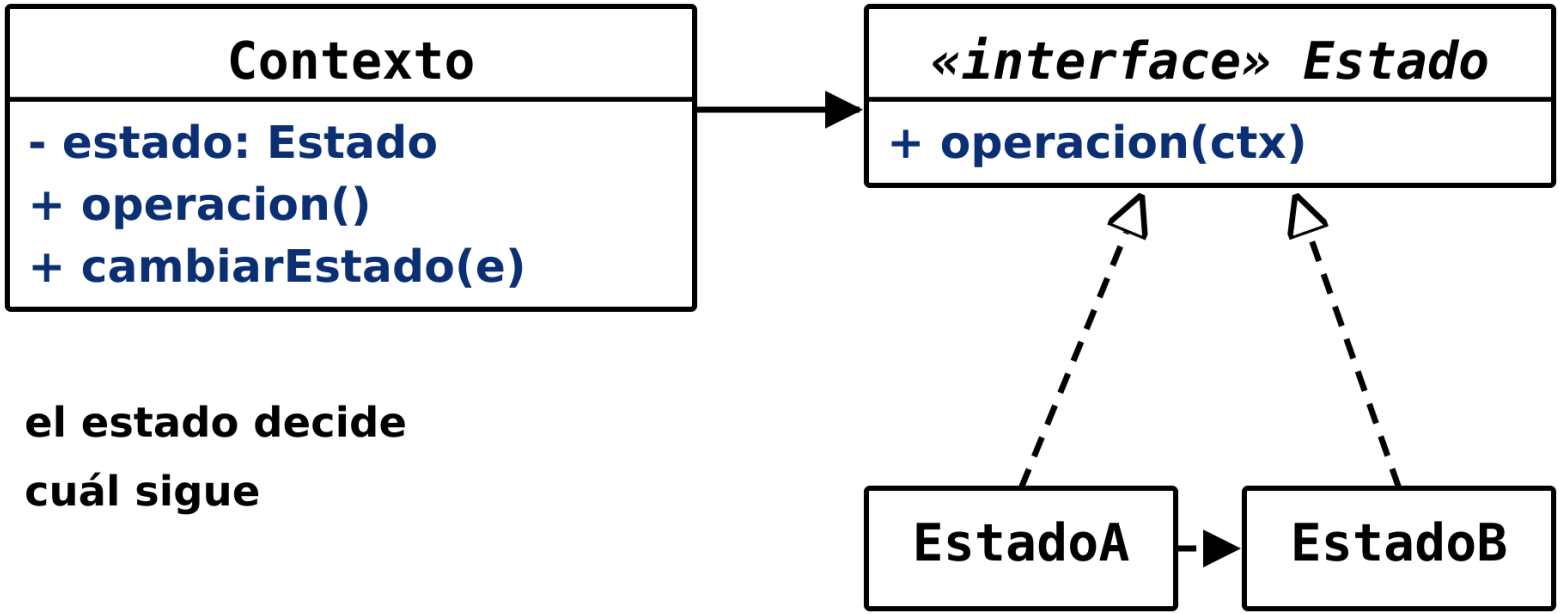
Si la acción se tiene que guardar, mostrar o deshacer, es un comando.

State: el problema



3 switch × 3 etapas, transiciones desparramadas

State: una clase por etapa



el estado decide

cuál sigue

State: el documento delega

```
public class Documento {
  private EstadoDocumento estado =
      new Borrador();
  public boolean publicar() {
    return estado.publicar(this);  // sin if
  }
  void cambiarEstado(EstadoDocumento e) {
    estado = e;
  }
}
```

State: la etapa decide

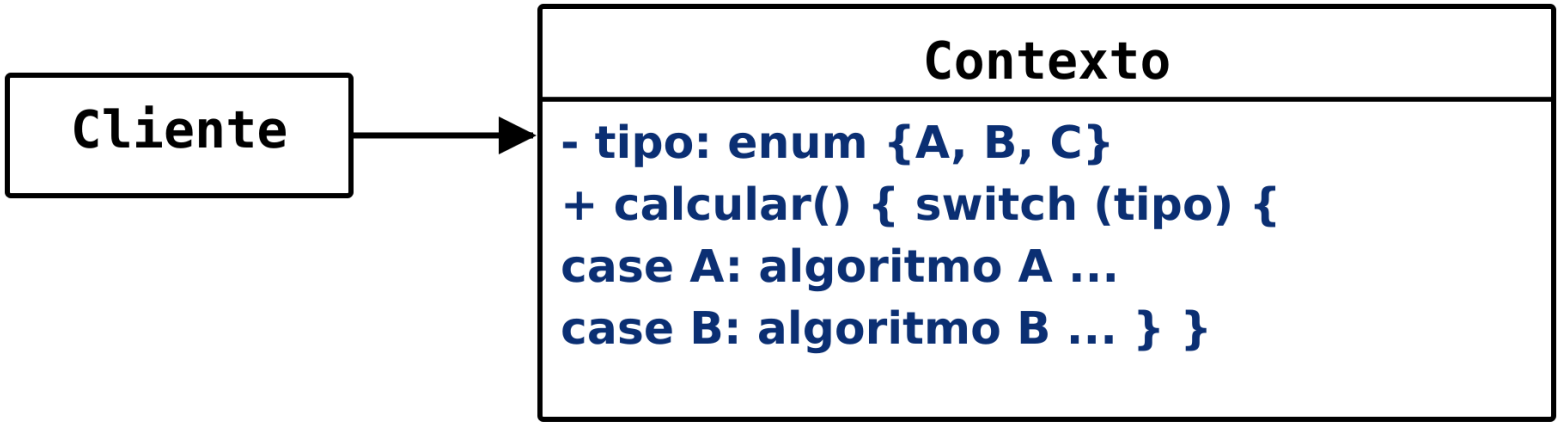
```
public class EnRevision
    implements EstadoDocumento {
  public boolean editar(Documento d, String t) {
    return false;
  }
  public boolean publicar(Documento d) {
    d.cambiarEstado(new Publicado());
    return true;
  }
}
```

State en el proyecto

- ¿Qué objeto pasa por etapas con reglas distintas en cada una?
- Los turnos, las fases de la partida, o una unidad: activa, aturdida, caída.

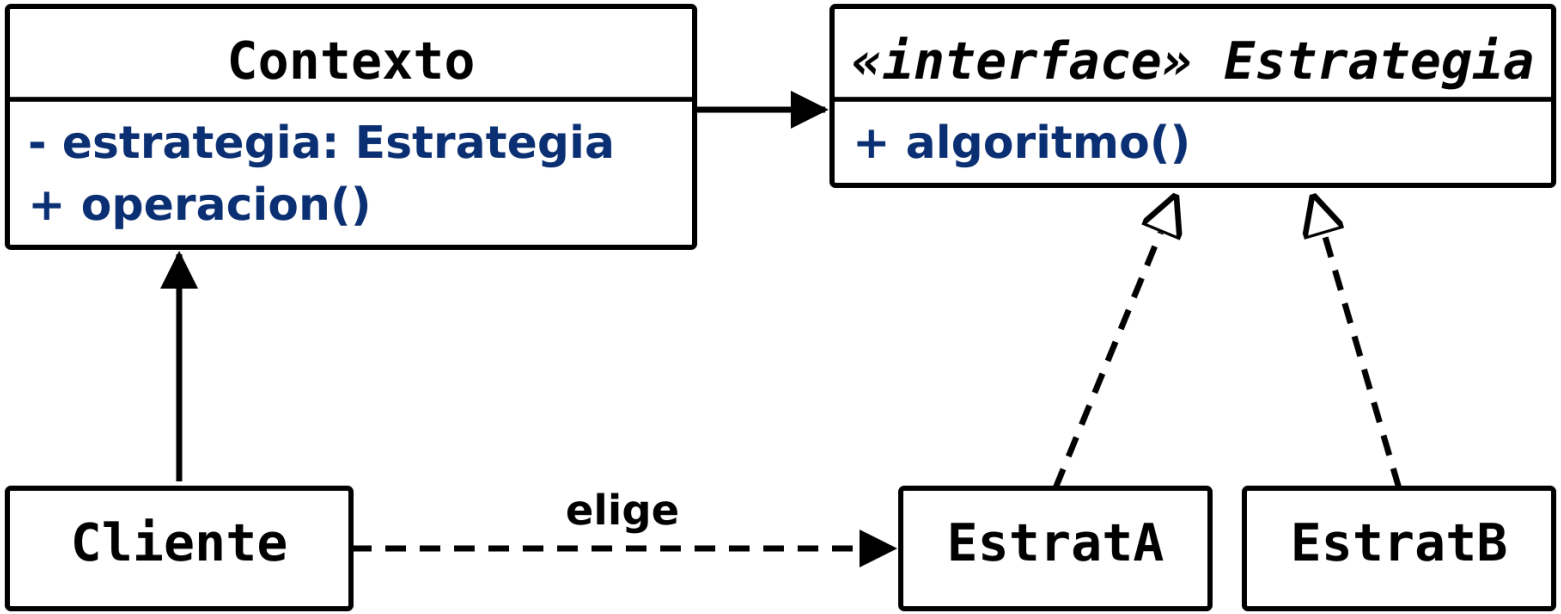
Un `switch` sobre la etapa en varios métodos es candidato a State.

Strategy: el problema



cada manera nueva abre la clase

Strategy: el cliente elige



Strategy: las estrategias

```
public interface EstrategiaEnvio {
  double costo(double kg, int km);
}

public class EnvioEstandar
    implements EstrategiaEnvio {
  public double costo(double kg, int km) {
    return 500 + 120 * kg;
  }
}
```

Strategy: el pedido delega

```
public double costoEnvio() {     // en Pedido
  return envio.costo(pesoKg, distanciaKm);
}

EstrategiaEnvio envio = new EnvioEstandar();
Pedido p = new Pedido(2.5, 30, envio);
p.costoEnvio();                          // 800
p.cambiarEnvio(new EnvioExpres());
p.costoEnvio();                          // 2240
```

Strategy en el proyecto

- ¿Qué tarea tiene varias maneras de hacerse, y quién elige cuál?
- La táctica de cada enemigo, o la dificultad, elegidas al armar el nivel.

El proyecto pide Observer y al menos uno de Command, State o Strategy, justificado.

State y Strategy: la diferencia

State

- En qué etapa está.
- Cambia desde adentro.
- Cada estado conoce al siguiente.

Strategy

- Cómo hace algo.
- Cambia desde afuera.
- Las estrategias no se conocen.

Si lo elige quien usa al objeto, es Strategy. Si cambia solo, es State.

En la biblioteca (1)

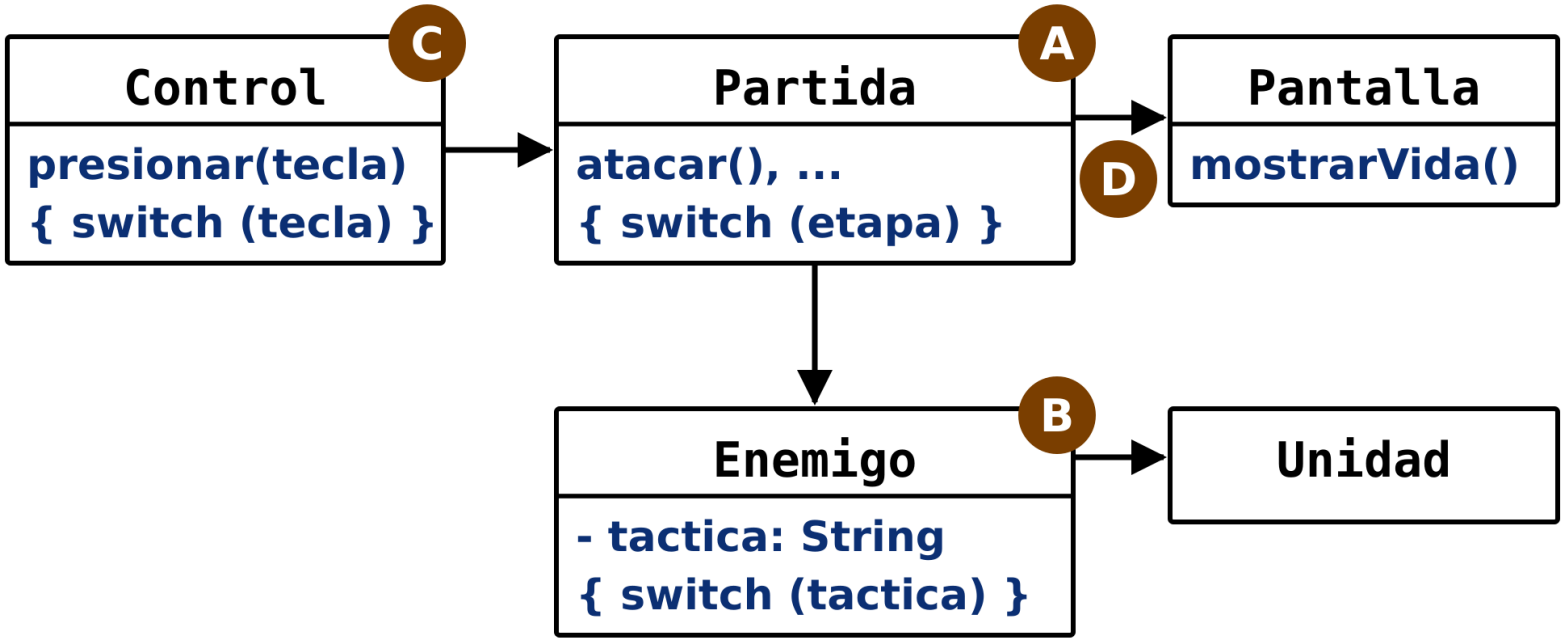
- **Observer**: un botón. **`addActionListener()`** registra, **`actionPerformed()`** avisa.
- **Command**: una **`Action`** de Swing sirve para un botón, un menú y un atajo.
- **Command**: cada **`Runnable`** de **`invokeLater()`** espera en una cola.

En la biblioteca (2)

- **Strategy**: `Collections.sort(lista, comparador)`. El criterio lo escribe quien llama.
- **Strategy**: un `LayoutManager` es la estrategia de un panel.
- **State**: un `Thread` nuevo, corriendo o terminado. `start()` funciona una sola vez.

Un botón es Observer, una `Action` es Command, un comparador es Strategy.

La actividad: cuatro problemas



La actividad de hoy

- **Parte 1**: qué patrón va en A, B, C y D, y por qué.
- **Parte 2**: aplicar tres de ellos, TODO por TODO.
- La salida tiene que quedar idéntica, registro incluido.
- Para quien termina antes: un orco cobarde y el cuarto patrón.